

**Cryptographic Strategies for Replay Attack Prevention: An Analysis
Using Timestamps and Nonces in CrypTool
(COMP5005)**

Submitted by:

KARTHIK ASHOK HONGUNTIKAR

University of Plymouth

Student ID: 10877925

karthik.honguntikar@postgrad.plymouth.ac.uk

Subject: Security Architectures and Cryptography

This is submitted to the University of Plymouth in partial fulfilment of the requirement for
the degree of MSc in cybersecurity

1. Introduction

Replay attacks represent a significant threat in secure communication systems, where an attacker (Oscar) intercepts and replays messages between two parties (Alice and Bob) multiple times. This report examines the capabilities of Digital Signatures (DS) and Message Authentication Codes (MAC) in detecting and preventing replay attacks. Although both methods authenticate the message and verify the sender's identity, they alone do not inherently prevent repeated transmissions of identical messages. Here, we analyse their limitations and propose enhanced solutions using timestamps and nonces to combat replay attacks. The report also includes implementation using the CrypTool application.

Section 1 – Question 1

Digital Signatures and Message Authentication Codes in Replay Attack Prevention

1. Digital Signatures (DS) Protection Against Replay Attack

Explanation of DS Mechanism:

A Digital Signature (DS) is created using a pair of cryptographic keys: a private key for signing and a public key for verification. Alice uses her private key to sign the message $x = \text{"Transfer \$1000 to Oscar"}$, producing $\text{auth}(x)$. Bob then uses Alice's public key to verify that $\text{auth}(x)$ is indeed from Alice and that the message has not been altered.

Replay Attack Analysis with DS:

In a replay attack, Oscar intercepts x and $\text{auth}(x)$ and sends them repeatedly to Bob. Since the DS on x is valid and does not change with repeated sends, Bob can verify each instance as authentic but has no way to tell if it's a repeat or the original message.

Conclusion:

Digital Signatures do not inherently protect against replay attacks. They provide message integrity and authentication, ensuring the message's origin and validity, but they cannot differentiate between an original and a replayed message.

2. Message Authentication Codes (MAC) Protection Against Replay Attack

Explanation of MAC Mechanism:

A Message Authentication Code (MAC) is generated using a symmetric key shared between Alice and Bob. Alice uses this shared key to create $\text{auth}(x)$ for $x = \text{"Transfer \$1000 to Oscar"}$. When Bob receives x and $\text{auth}(x)$, he verifies it with the shared key, confirming both message integrity and authenticity.

Replay Attack Analysis with MAC:

Since MACs are created from the same key for identical messages, Oscar will produce the same $\text{auth}(x)$ value for each replayed instance of x . Bob has no way to distinguish between the initial message and replayed ones if they appear identical, as the MAC remains consistent across identical messages with the same key.

Section 2 - Question 2

Solution 1: Using Timestamps

Description:

Alice can include a timestamp when sending each message. The timestamp records the exact time at which x (the message) was created. Bob then checks the timestamp to ensure that each received message is recent and discards messages with outdated timestamps.

How It Works:

1. Alice's Actions: Alice sends $x = \text{"Transfer \$1000 to Oscar"}$ along with the current timestamp t and a computed $\text{auth}(x \parallel t)$ (the authentication tag of the message and timestamp).
2. Bob's Actions: Upon receiving x , t , and $\text{auth}(x \parallel t)$, Bob verifies the authenticity of the message and checks if the timestamp is within an acceptable range (e.g., within the last 5 minutes).
3. Oscar's Replay Attack Prevention: If Oscar resends the message after the allowed timeframe, the timestamp t will be considered stale, and Bob will reject the replayed message.

Implementation Steps in CrypTool or a similar cryptographic tool:

- Step 1: Create a message with the text and a current timestamp.
- Step 2: Generate $\text{auth}(x \parallel t)$ using a DS or MAC, depending on the shared secret or public-private key setup.
- Step 3: Verify the $\text{auth}(x \parallel t)$ and check the timestamp validity.

Solution 2: Using Nonces

Description:

A nonce is a unique, one-time number that Alice includes with each message. This nonce ensures that each message is uniquely identifiable. Alice and Bob maintain a list of used nonces to prevent replay.

How It Works:

1. Alice's Actions: Alice generates a unique nonce n for each session or transaction. She sends $x = \text{"Transfer \$1000 to Oscar"}$ along with n and computes $\text{auth}(x \parallel n)$.
2. Bob's Actions: Upon receiving x , n , and $\text{auth}(x \parallel n)$, Bob verifies the authenticity and checks if n has been used before. If n is new, he accepts the message; if n is repeated, he rejects it.
3. Oscar's Replay Attack Prevention: Oscar cannot reuse an old nonce, as it would be detected by Bob.

Implementation Steps in CrypTool or a similar cryptographic tool:

- Step 1: Generate a unique nonce for each message.
- Step 2: Concatenate the nonce with the message and compute $\text{auth}(x \parallel n)$ using DS or MAC.
- Step 3: Verify the $\text{auth}(x \parallel n)$ at Bob's end, ensuring the nonce has not been used.

1. Solution Using Timestamps

Mechanism:

Alice includes a timestamp with each message. This timestamp specifies the exact time when the message was sent and is valid only within a predetermined time window. Bob checks whether the timestamp falls within this acceptable timeframe; if the timestamp is outdated, Bob identifies the message as a replay and rejects it.

Process Flow:

- Step 1: Alice sends a message, $x = \text{"Transfer \$1000 to Oscar"}$, along with a timestamp t , and computes $\text{auth}(x \parallel t)$.
- Step 2: Bob receives x , t , and $\text{auth}(x \parallel t)$. He verifies the authenticity of $\text{auth}(x \parallel t)$ and checks if the timestamp t is within an acceptable range.
- Step 3: If Oscar replays the message with the same timestamp after the valid period, Bob detects that it is outdated and rejects the message.

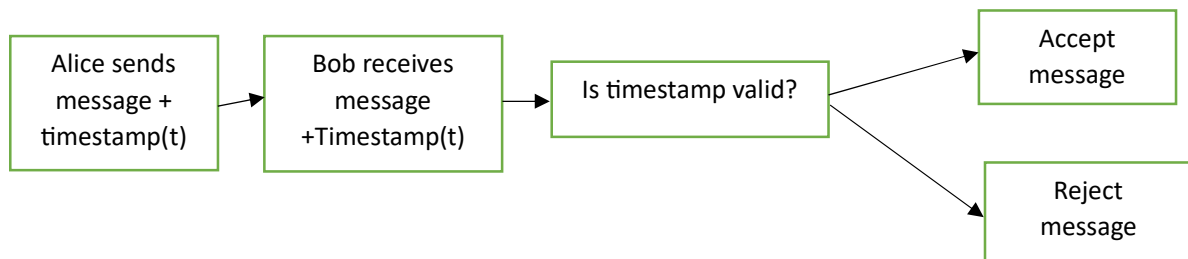
Implementation with CrypTool:

- In CrypTool, you can append a timestamp to the message x before computing the DS or MAC.
- During verification, CrypTool allows setting conditions to check the timestamp's validity.
- If the timestamp is outside the acceptable range, CrypTool flags the message for rejection.

Flowchart:

Alice → Sends Message + Timestamp → Bob

Oscar → Replays Message with Old Timestamp → Bob (Rejected)



1. Alice generates x with t and computes $\text{auth}(x \parallel t)$.
2. Bob receives x , t , and $\text{auth}(x \parallel t)$, checks authenticity, and verifies that t is within an acceptable timeframe.
3. Replay (Oscar) with an outdated timestamp is detected and rejected by Bob.

2. Solution Using Nonces

Mechanism:

Alice generates a unique nonce (a random identifier) for each transaction. Each nonce ensures that every message is distinct, even if the content of x remains the same. Bob keeps a record of each nonce received. If he detects a duplicate nonce, he recognizes it as a replay and discards the message.

Process Flow:

- Step 1: Alice sends x = "Transfer \$1000 to Oscar" along with a unique nonce n , generating $\text{auth}(x \parallel n)$.
- Step 2: Bob receives x , n , and $\text{auth}(x \parallel n)$. He verifies the authenticity and checks if n has been used previously.
- Step 3: If Oscar attempts to resend x with the same nonce n , Bob identifies it as a duplicate and rejects the message.

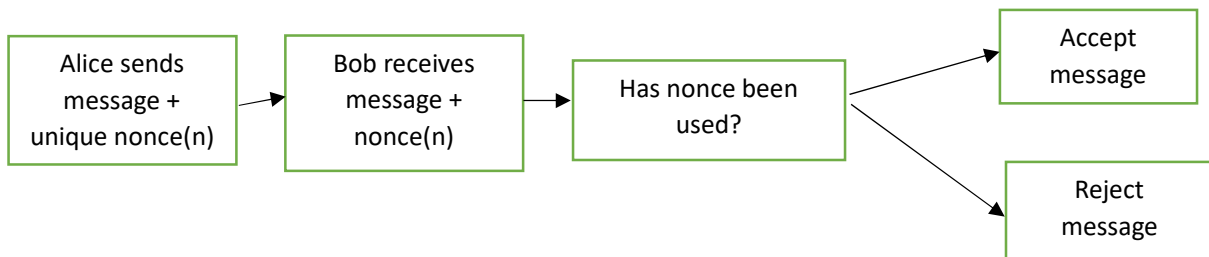
Implementation with CrypTool:

- In CrypTool, add a nonce to each message before computing $\text{auth}(x \parallel n)$.
- During verification, CrypTool can store each nonce in a log, flagging any repeated nonce for rejection.

Flowchart:

Alice → Sends Message + Unique Nonce → Bob

Oscar → Replays Message with Same Nonce → Bob (Rejected)



1. Alice generates x with a unique n and computes $\text{auth}(x \parallel n)$.
2. Bob receives x , n , and $\text{auth}(x \parallel n)$, verifies authenticity, and checks if n has been used before.
3. Replay (Oscar) with the same nonce is detected as a duplicate and rejected by Bob.

4. Implementation Evidence Using CrypTool

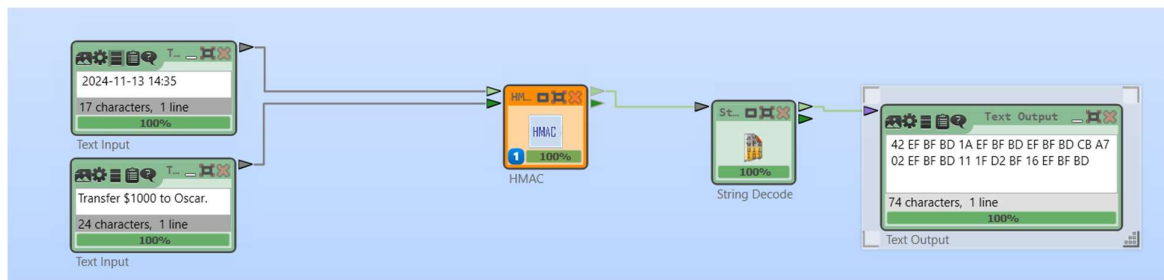
1. Timestamp Inclusion:

Purpose: The timestamp ensures that each message has a valid time window, so if an attacker tries to replay an old message, it will be detected and rejected as "expired."

Implementation Steps:

1. Text Input (Message): Enter your primary message, e.g., "Transfer \$1000 to Oscar".
2. Text Input (Timestamp): Manually input the current timestamp, e.g., "2024-11-13 14:35".
3. HMAC Generation: Use the Concatenate component to combine the message and timestamp, then pass it through an HMAC component to generate an authentication code. This code will validate both the message and timestamp.

4. Verification: On the recipient's end, the timestamp should be verified within an acceptable range. If the timestamp is outdated, the message is rejected as a replay attempt.



2. Nonce Inclusion:

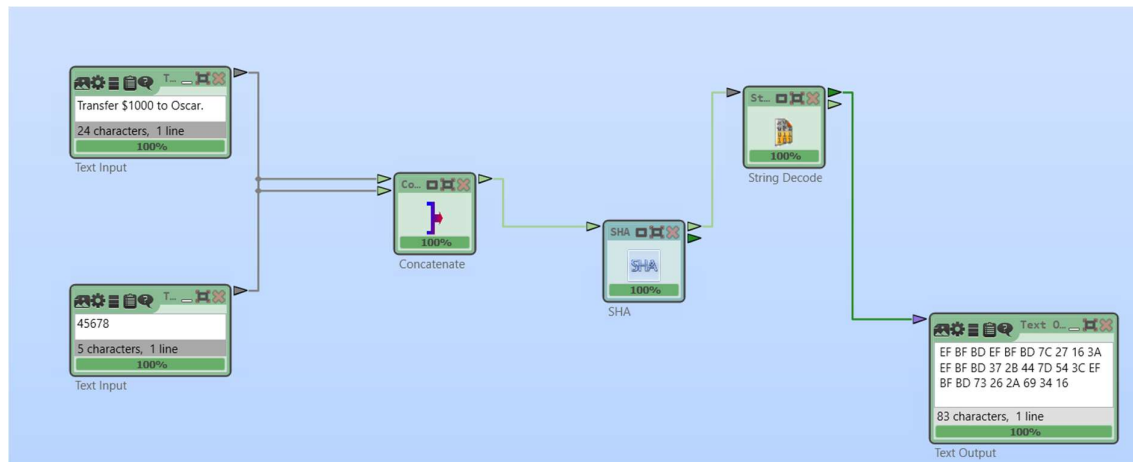
Purpose: A nonce is a unique identifier added to each message to ensure that no two messages are identical, preventing replay attacks.

Implementation Steps:

1. Text Input (Message): Enter the main message as above.
2. Random Number Generator: Use this component to generate a unique nonce for each message.
3. Concatenate: Combine the message and nonce.
4. SHA or HMAC: Pass the concatenated result through an SHA or HMAC component to secure the message with its unique nonce.

SHA (Secure Hash Algorithm) is used here to create a unique hash of the combined message and nonce. It provides a tamper-evident fingerprint, ensuring data integrity. If even a small part of the message or nonce is altered, the resulting hash will differ significantly, helping to detect tampering or replay attempts. SHA's one-way nature makes it ideal for securely verifying data authenticity without revealing the original content.

5. Verification: At the recipient's end, store each nonce in a log or database. If a duplicate nonce is detected, the message is flagged as a replay and rejected.



5. Conclusion

Digital Signatures (DS) and Message Authentication Codes (MAC) are essential tools for verifying message authenticity and ensuring data integrity. However, on their own, they do not inherently protect against replay attacks, where identical messages can be intercepted and resent by an attacker. By incorporating timestamps and nonces, we can introduce an additional layer of security that allows the recipient to identify and reject repeated messages. The use of timestamps ensures each message is recent, while nonces provide a unique identifier for each transmission. Together, these techniques effectively mitigate replay risks.

The CrypTool application has proven valuable in demonstrating this enhanced approach, showing how timestamps and nonces can be seamlessly integrated with DS or MAC to secure communications. This method provides a robust security framework, especially critical in financial transactions and other sensitive data exchanges, where unauthorised repeated actions could lead to significant consequences. The combined use of timestamps and nonces establishes a more resilient system, preventing replay attacks and strengthening the integrity of secure communications.

6. References:

W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. Upper Saddle River, NJ, USA: Pearson, 2017.

National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, Aug. 2015. Available: <https://doi.org/10.6028/NIST.FIPS.180-4>

B. Schneier, *Applied Cryptography: Protocols, Algorithms, and Source Code in C*, 2nd ed. New York, NY, USA: Wiley, 1996.

CrypTool Team, *CrypTool 2 User Manual*. [Online]. Available: <https://www.cryptool.org/en/>

W. Diffie and M. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, Nov. 1976, doi: 10.1109/TIT.1976.1055638.